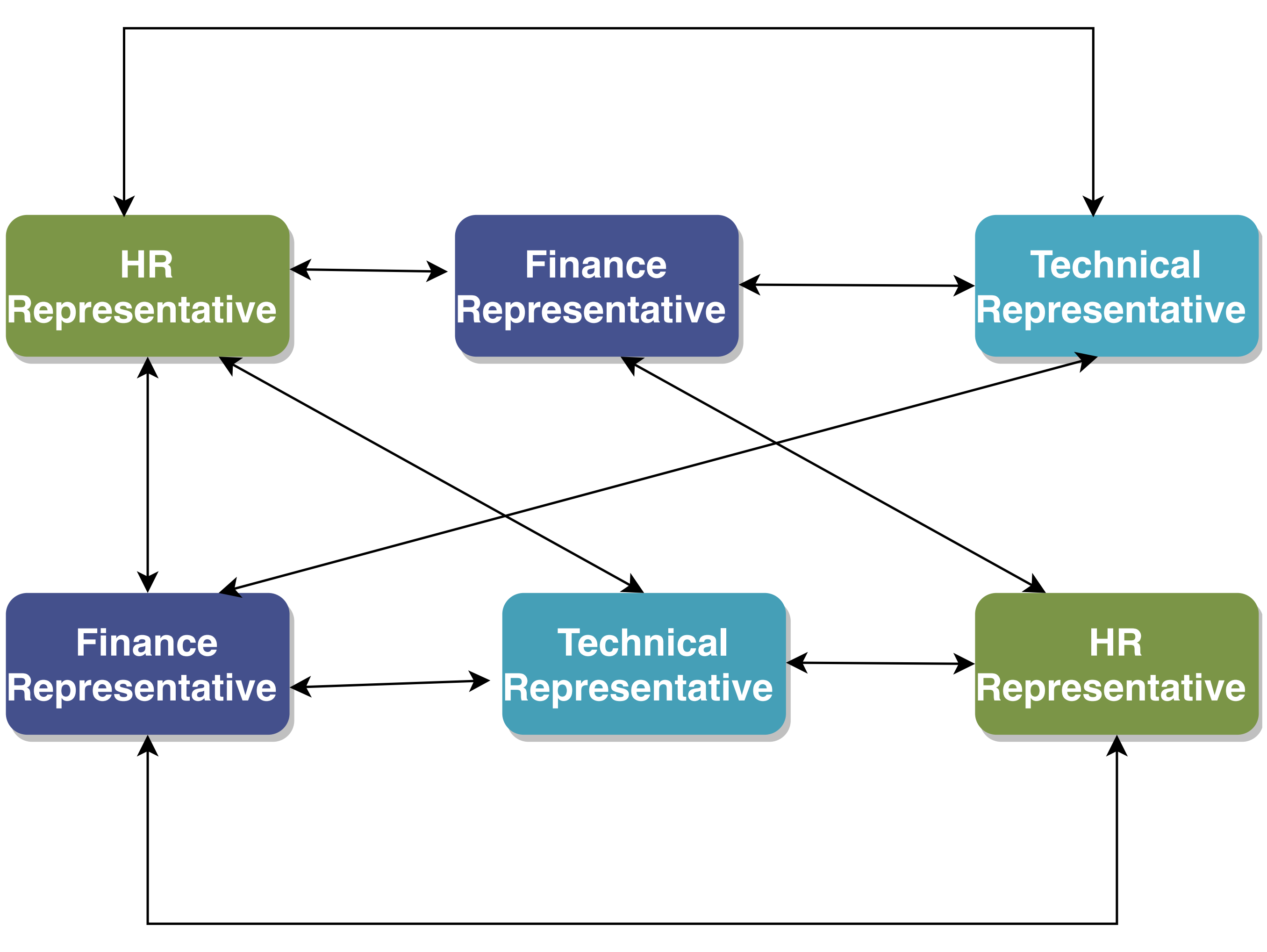
Mediator Pattern

The Mediator pattern serves as a centralized communication medium for various objects in a system. The pattern seeks to reduce coupling, increase system scalability, and facilitate easier understanding and maintenance by minimizing direct communication between objects or classes.

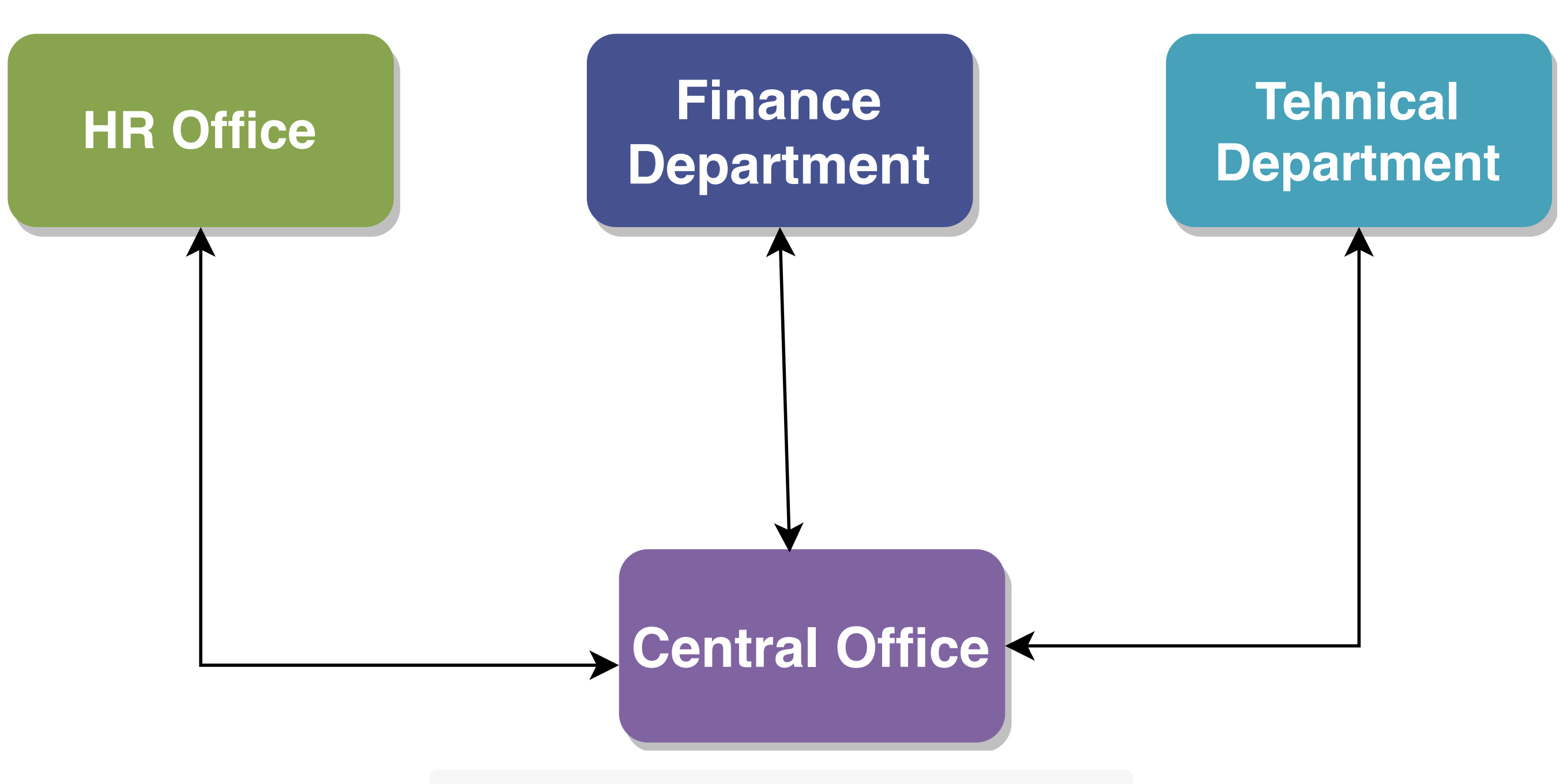
For instance, within an office setting, communication between different departments (such as HR, Finance, and Technical) is often needed. Direct communication between departments for every minor query or update can result in an unorganized and inefficient workflow. Suppose the Technical department requires data for a new project from HR and Finance. In that case, handling this communication can get complicated and time-consuming. The image below illustrates the scenario:



Mediator Pattern - Office Communication Problem

Implementing the Mediator pattern in this scenario involves setting up a central communication office or a mediator. This office serves as the primary point of contact for interdepartmental communication. This office receives all requests and information and forwards it to the relevant department.

- The central office receives a request from the technical department.
- After processing this request and obtaining the required data from HR and Finance, the central office sends the information back to the technical department.
- In this manner, the mediator serves as a medium for communication between all departments, cutting down on direct lines of communication and improving workflow efficiency by decreasing error rates.



Mediator Pattern - Office Communication Solution

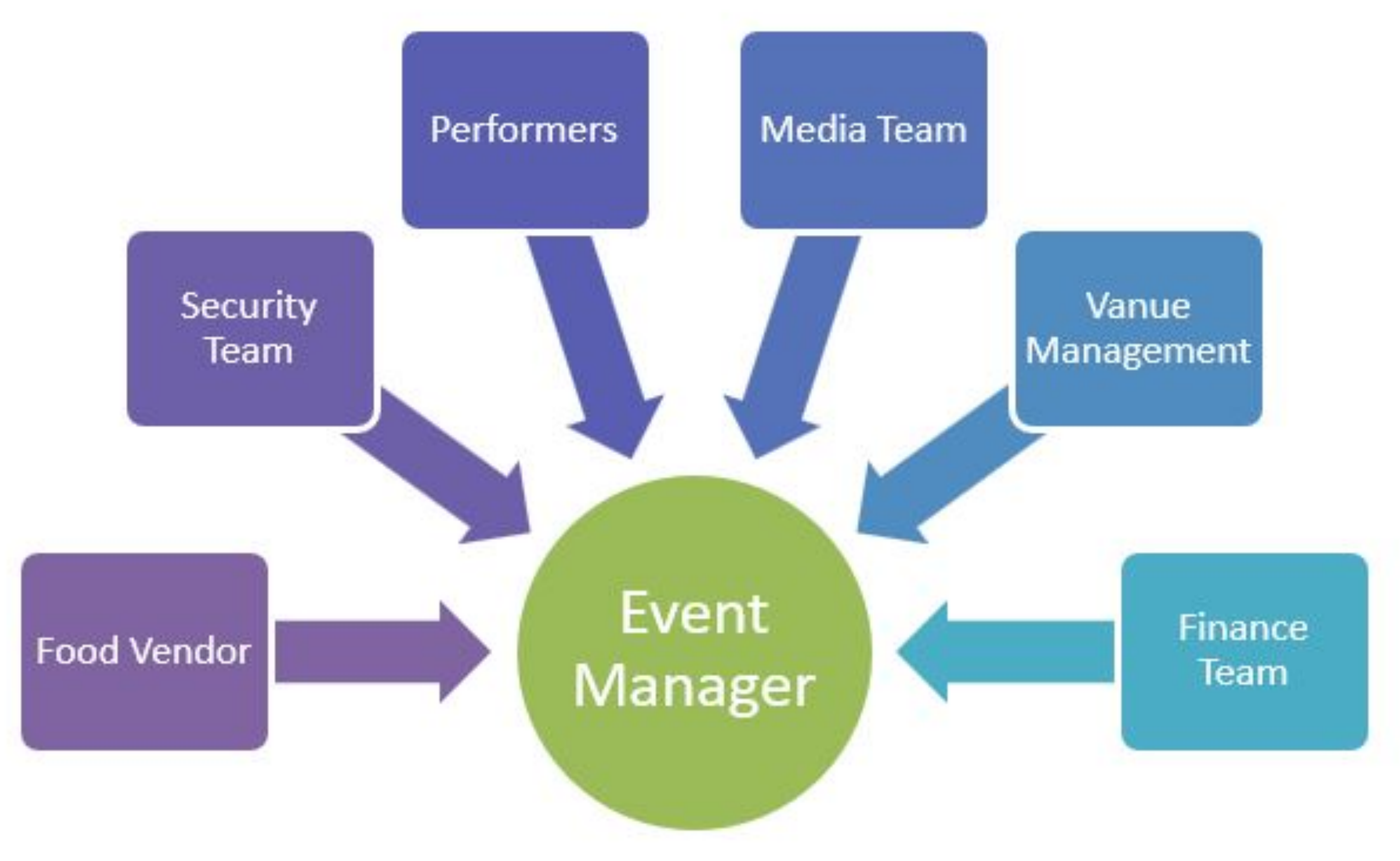
Real-World Example

Consider the role of event organizers in the planning of a large event such as a music festival. Many parties are involved in the event, including vendors, venue management, security, and performers.

Direct Communication Issue: The process would soon become unmanageable if all parties communicated with each other directly for every issue. It would be chaotic, for example, if each vendor had to communicate directly with performers and security to meet their needs.

Solution for Mediator: An event planner serves as a mediator. They arrange for all correspondence and arrangements between the various stakeholders. The event organizer receives updates and needs from vendors, security, performers, and venue management and forwards them to the relevant parties. This streamlines and expedites the process by centralizing communication.

For better understanding, this is illustrated in the image below:

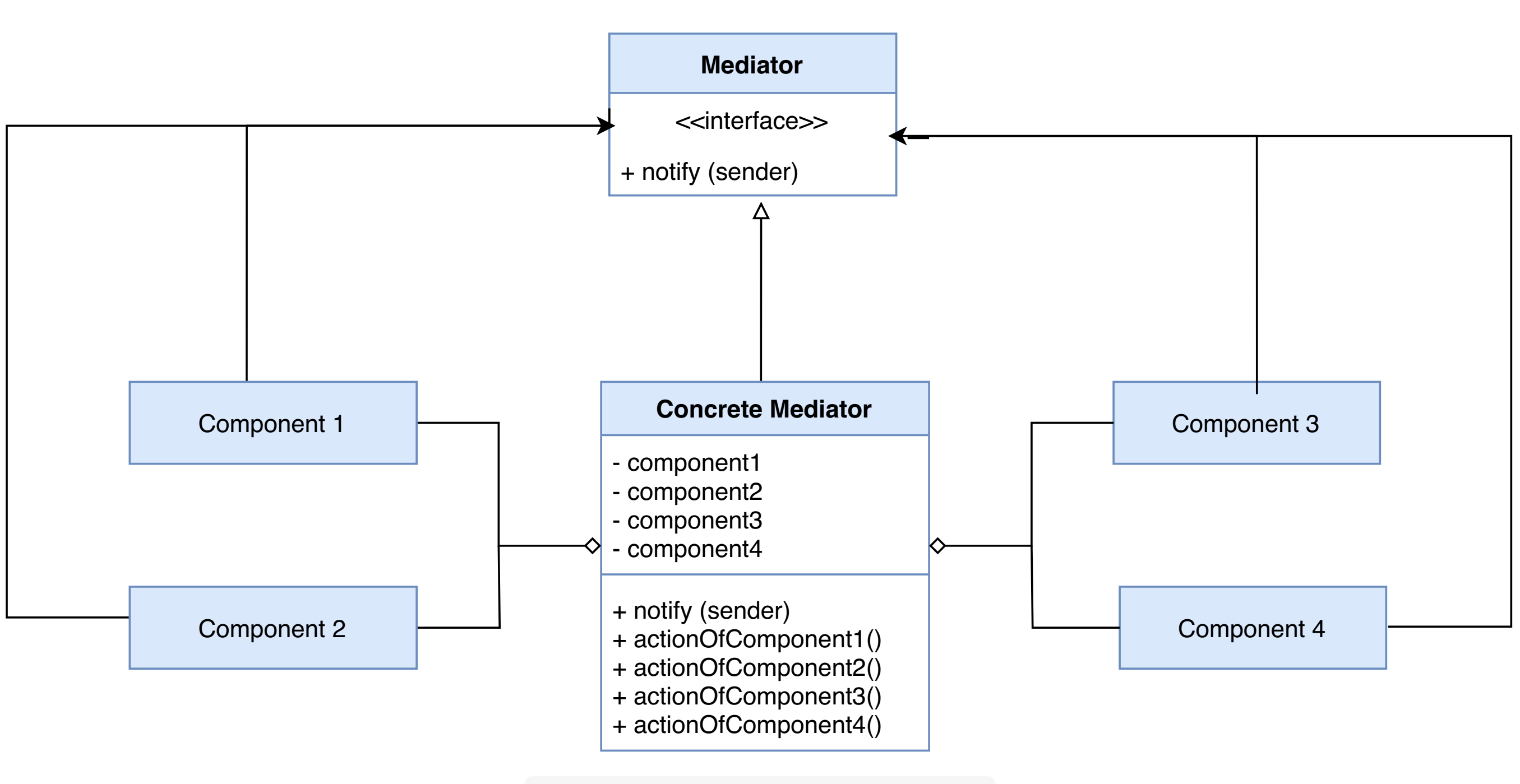


Mediator Pattern -Real-World Example

Structure of Mediator Pattern

- **Mediator Interface**: An interface that defines methods for communicating with component objects.
- **Concrete Mediator**: Coordinates communication between different components and implements the mediator interface. It is aware of all of the components involved and facilitates their interaction.
- **Component**: Defines different classes that interact with one another. Instead of communicating directly, they use the mediator object.

Through the mediator, components communicate with one another. They communicate with the mediator, who forwards their messages to the relevant component. The mediator centralizes complex communication and control logic between related objects.



Mediator Pattern - Class Diagram

Implementation of Mediator Pattern

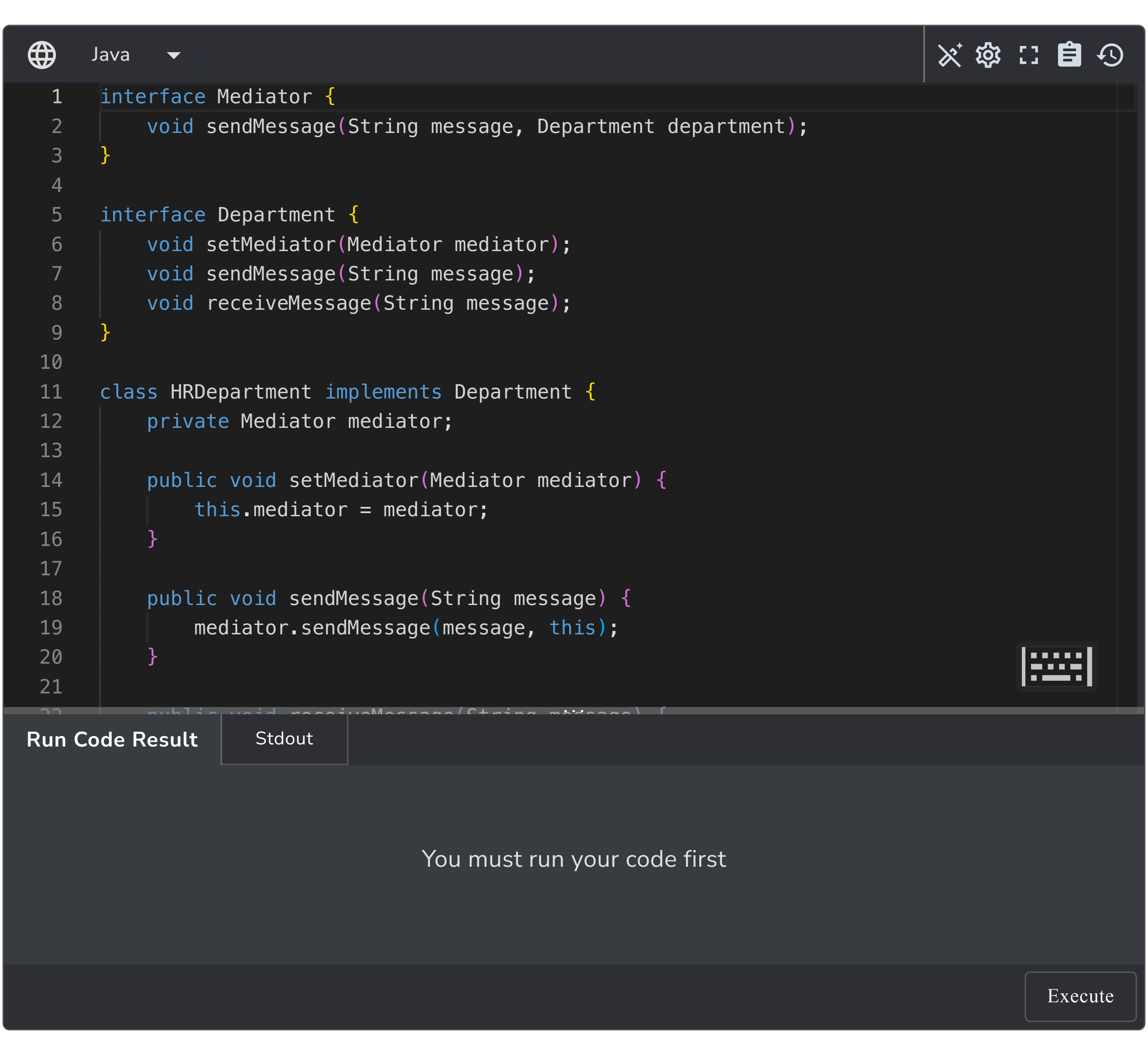
Recap the office example discussed in the previous section. Departments such as HR, Finance, and Technical need to communicate with one another. Direct communication between departments can be chaotic and ineffective.

Let's implement its solution through mediator pattern.

```
// Mediator Interface
interface DepartmentMediator {
    communicate(message: String, department: Department)
}

// Concrete Mediator
class OfficeMediator implements DepartmentMediator {
    HRDepartment hr
    FinanceDepartment finance
    TechnicalDepartment technical

    setHRDepartment(department: HRDepartment) {
        hr = department
    }

    setFinanceDepartment(department: FinanceDepartment) {
        finance = department
    }

    setTechnicalDepartment(department: TechnicalDepartment) {
        technical = department
    }

    communicate(message: String, department: Department) {
        if (department == hr) {
            finance.receiveMessage(message)
            technical.receiveMessage(message)
        } else if (department == finance) {
            // similar logic
        } else if (department == technical) {
            // similar logic
        }
    }
}

// Department Interface
interface Department {
    setMediator(mediator: DepartmentMediator)
    sendMessage(message: String)
    receiveMessage(message: String)
}

// Concrete Departments
class HRDepartment implements Department {
    DepartmentMediator mediator

    setMediator(mediator: DepartmentMediator) {
        this.mediator = mediator
    }

    sendMessage(message: String) {
        mediator.communicate(message, this)
    }

    receiveMessage(message: String) {
        // Handle received message
    }
}

// Similar classes for FinanceDepartment and TechnicalDepartment

// Client Code
main() {
    mediator = new OfficeMediator()
    hr = new HRDepartment()
    finance = new FinanceDepartment()
    technical = new TechnicalDepartment()

    mediator.setHRDepartment(hr)
    mediator.setFinanceDepartment(finance)
    mediator.setTechnicalDepartment(technical)

    hr.setMediator(mediator)
    finance.setMediator(mediator)
    technical.setMediator(mediator)

    hr.sendMessage("HR update for all departments")
}
```

- **OfficeMediator** (**Concrete Mediator**): Helps in communication between different departments. It has methods to set department references and a `communicate` method to route messages.
- **Department** (**Interface**): Each department in the office has a reference to a mediator for message exchange.**HRDepartment, FinanceDepartment, TechnicalDepartment** (**Concrete Departments**): Implement the Department interface. Each department uses the mediator to send messages to other departments.
- **Client**: Sets up the mediator and departments, and initiates communication.

Implementation



```
interface Mediator {
    void sendMessage(String message, Department department);
}

interface Department {
    void setMediator(Mediator mediator);
    void sendMessage(String message);
    void receiveMessage(String message);
}

class HRDepartment implements Department {
    private Mediator mediator;

    public void setMediator(Mediator mediator) {
        this.mediator = mediator;
    }

    public void sendMessage(String message) {
        mediator.sendMessage(message, this);
    }

```

You must run your code first

Application of Mediator Pattern

The Mediator pattern is widely used to manage complex communications and interactions in a variety of fields. Primary applications consist of:

- **User Interface (UI) Development**: Reducing the complexity of communication between different UI elements in complex graphical user interfaces, like buttons, menus, and dialogs.
- **Chat applications**: Acting as the main mediator for messages in chat systems where users communicate on a single platform.
- **Control Systems**: In automated control systems, a centralized system coordinates actions among various entities, like air traffic control.
- **System integration**: facilitating communication and data exchange between various systems or software components in enterprise applications.
- **Game Development**: Managing interactions between game components such as characters, AI, and game levels.

Reduced component coupling and centralized control simplify interactions, increase system manageability, and enable scalability for every application.

Pros and Cons

Pros	Cons
Reduces Dependencies Reduces the dependencies between communicating objects or classes.	**Complex Mediator** The mediator can become overly complex as it centralizes control.
Centralizes Control Simplifies communication by centralizing it in one object or class.	**Single Point of Failure** The mediator becomes a critical part of the system, and its failure can impact the entire system.
Simplifies Maintenance It is easier to maintain and modify communication logic in one place.	**Performance Bottleneck** All communication through the mediator may lead to performance issues in large systems.
Promotes Loose Coupling Helps in building systems where components are loosely coupled.	**Debugging Challenges** Debugging can be difficult if the mediator's logic is too complex.

The Mediator pattern is primarily used to manage complex system communications. It performs best when it is important to minimize direct coupling between components. By centralized communication, the system becomes easier to understand and maintain. But, it's crucial to be aware of the complexity it may bring along with the possibility of single points of failure and performance bottlenecks. When used properly, the Mediator pattern can result in a code architecture that is scalable, clean, and well-organized.

← Previous
Iterator Pattern

Next →
Memento Pattern

☐ Mark as Completed